Perform a SQL Query

Google Cybersecurity Certificate | SQL Foundations Lab
Daniel Dela Dzikunu | May 28, 2026

Project Description

This lab activity demonstrates foundational SQL querying skills within a simulated security-analyst workflow. Working inside a MariaDB shell connected to the **organization** database, the objective was to retrieve device and login data relevant to an endpoint-update audit and a login-anomaly investigation. The lab covered three core SQL techniques: selecting specific columns, retrieving all columns using the wildcard `*` operator, and sorting result sets with the `ORDER BY` keyword.

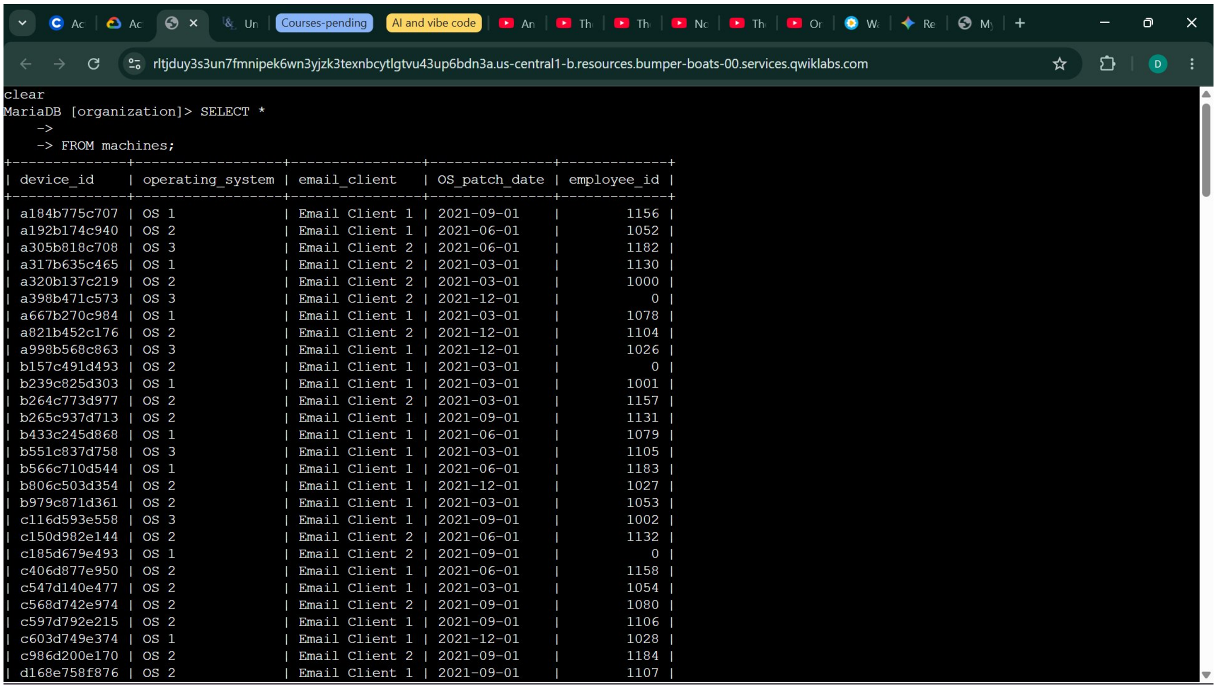
Task 1 — Retrieve Employee Device Data

The **machines** table stores device identifiers, operating systems, email clients, OS patch dates, and employee IDs. Three queries were executed to progressively narrow the columns returned.

Step 1 – Select all columns from the machines table

The wildcard `SELECT *` query returns every column and all 200 rows. This gives a complete picture of the device inventory before applying any column filters.

```
SELECT *
FROM machines;
```



```
clear
MariaDB [organization]> SELECT *
    ->
    -> FROM machines;
```

device_id	operating_system	email_client	OS_patch_date	employee_id
a184b775c707	OS 1	Email Client 1	2021-09-01	1156
a192b174c940	OS 2	Email Client 1	2021-06-01	1052
a305b818c708	OS 3	Email Client 2	2021-06-01	1182
a317b635c465	OS 1	Email Client 2	2021-03-01	1130
a320b137c219	OS 2	Email Client 2	2021-03-01	1000
a398b471c573	OS 3	Email Client 2	2021-12-01	0
a667b270c984	OS 1	Email Client 1	2021-03-01	1078
a821b452c176	OS 2	Email Client 2	2021-12-01	1104
a998b568c863	OS 3	Email Client 1	2021-12-01	1026
b157c491d493	OS 2	Email Client 1	2021-03-01	0
b239c825d303	OS 1	Email Client 1	2021-03-01	1001
b264c773d977	OS 2	Email Client 2	2021-03-01	1157
b265c937d713	OS 2	Email Client 1	2021-09-01	1131
b433c245d868	OS 1	Email Client 1	2021-06-01	1079
b551c837d758	OS 3	Email Client 1	2021-03-01	1105
b566c710d544	OS 1	Email Client 1	2021-06-01	1183
b806c503d354	OS 2	Email Client 1	2021-12-01	1027
b979c871d361	OS 2	Email Client 1	2021-03-01	1053
c116d593e558	OS 3	Email Client 1	2021-09-01	1002
c150d982e144	OS 2	Email Client 2	2021-06-01	1132
c185d679e493	OS 1	Email Client 2	2021-09-01	0
c406d877e950	OS 2	Email Client 1	2021-06-01	1158
c547d140e477	OS 2	Email Client 1	2021-03-01	1054
c568d742e974	OS 2	Email Client 2	2021-09-01	1080
c597d792e215	OS 2	Email Client 1	2021-09-01	1106
c603d749e374	OS 1	Email Client 1	2021-12-01	1028
c986d200e170	OS 2	Email Client 2	2021-09-01	1184
d168e758f876	OS 2	Email Client 1	2021-09-01	1107

*Figure 1 – Full machines table output (SELECT *)*

Step 2 – Select device_id and email_client only

Restricting the output to two columns makes it easier to audit which email client is deployed on each device. The third row shows **Email Client 2**.

```
SELECT device_id, email_client
FROM machines;
```

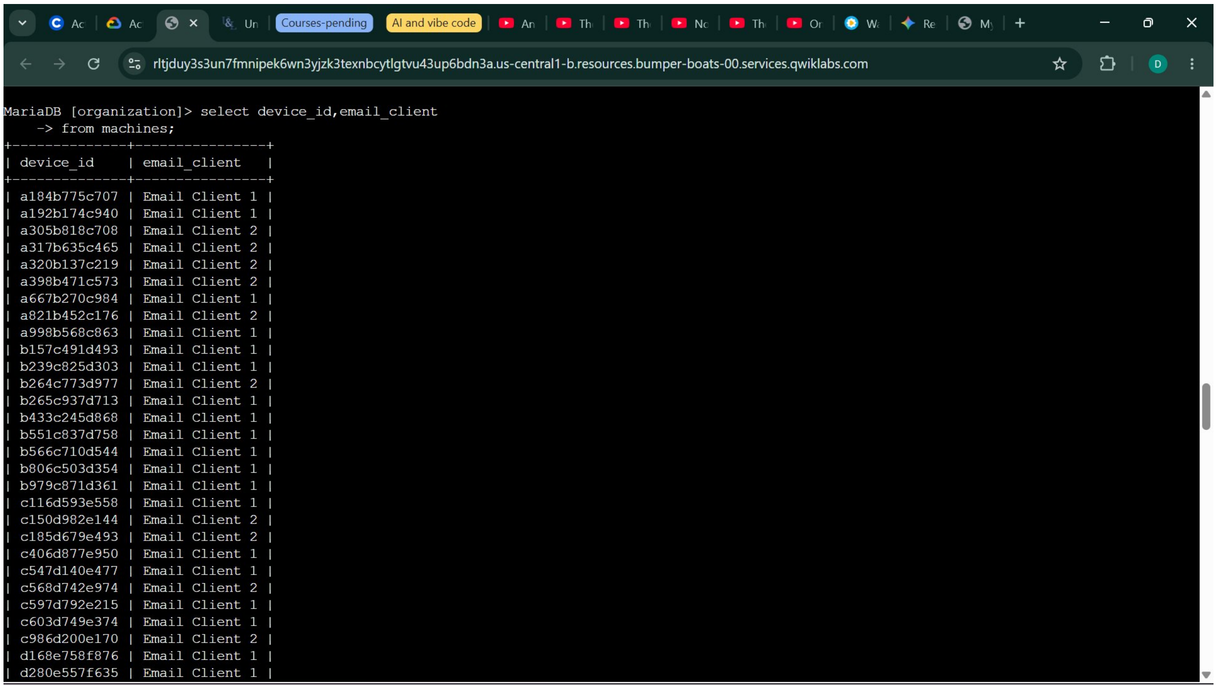
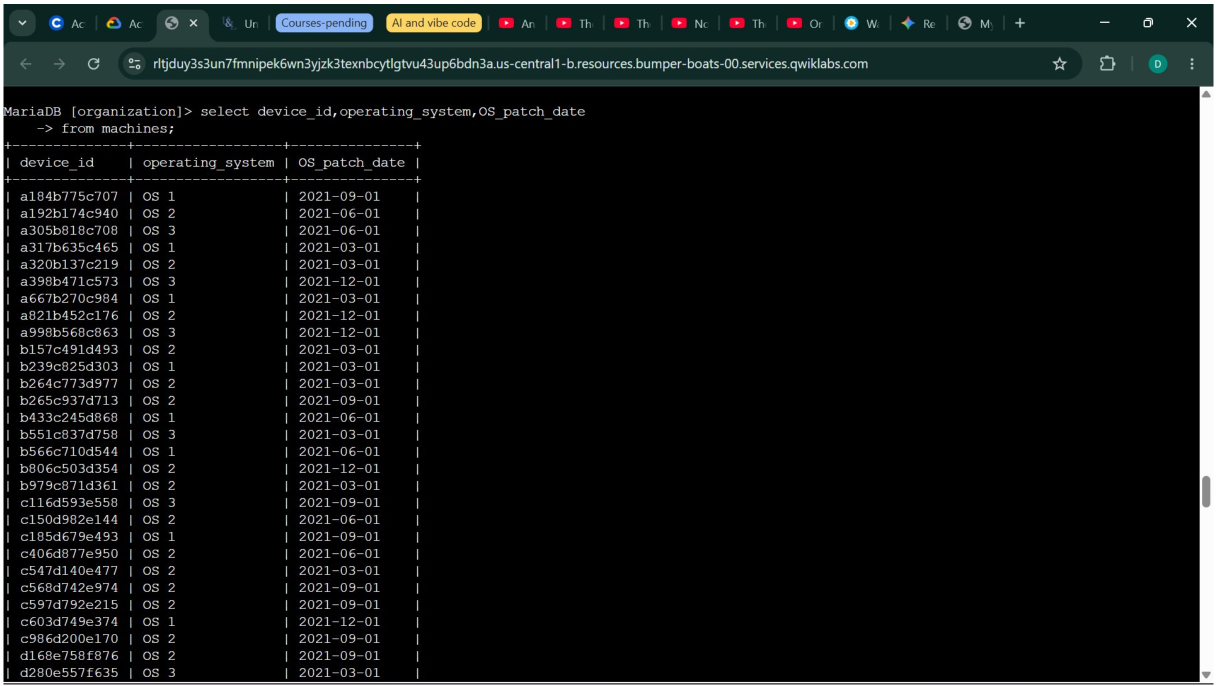


Figure 2 – device_id and email_client columns

Step 3 – Select device_id, operating_system, and OS_patch_date

This query surfaces patch currency per device. The first entry — device `a184b775c707` running OS 1 — has a patch date of **2021-09-01**, indicating it has not been patched in over four years and is a priority for the update cycle.

```
SELECT device_id, operating_system, OS_patch_date
FROM machines;
```



```
MariaDB [organization]> select device_id,operating_system,OS_patch_date
    -> from machines;
+-------------+------------------+---------------+
| device_id   | operating_system | OS_patch_date |
+-------------+------------------+---------------+
| a184b775c707 | OS 1            | 2021-09-01    |
| a192b174c940 | OS 2            | 2021-06-01    |
| a305b818c708 | OS 3            | 2021-06-01    |
| a317b635c465 | OS 1            | 2021-03-01    |
| a320b137c219 | OS 2            | 2021-03-01    |
| a398b471c573 | OS 3            | 2021-12-01    |
| a667b270c984 | OS 1            | 2021-03-01    |
| a821b452c176 | OS 2            | 2021-12-01    |
| a998b568c863 | OS 3            | 2021-12-01    |
| b157c491d493 | OS 2            | 2021-03-01    |
| b239c825d303 | OS 1            | 2021-03-01    |
| b264c773d977 | OS 2            | 2021-03-01    |
| b265c937d713 | OS 2            | 2021-09-01    |
| b433c245d868 | OS 1            | 2021-06-01    |
| b551c837d758 | OS 3            | 2021-03-01    |
| b566c710d544 | OS 1            | 2021-06-01    |
| b806c503d354 | OS 2            | 2021-12-01    |
| b979c871d361 | OS 2            | 2021-03-01    |
| c116d593e558 | OS 3            | 2021-09-01    |
| c150d982e144 | OS 2            | 2021-06-01    |
| c185d679e493 | OS 1            | 2021-09-01    |
| c406d877e950 | OS 2            | 2021-06-01    |
| c547d140e477 | OS 2            | 2021-03-01    |
| c568d742e974 | OS 2            | 2021-09-01    |
| c597d792e215 | OS 2            | 2021-09-01    |
| c603d749e374 | OS 1            | 2021-12-01    |
| c986d200e170 | OS 2            | 2021-09-01    |
| d168e758f876 | OS 2            | 2021-09-01    |
| d280e557f635 | OS 3            | 2021-03-01    |
```

Figure 3 – device_id, operating_system, and OS_patch_date columns

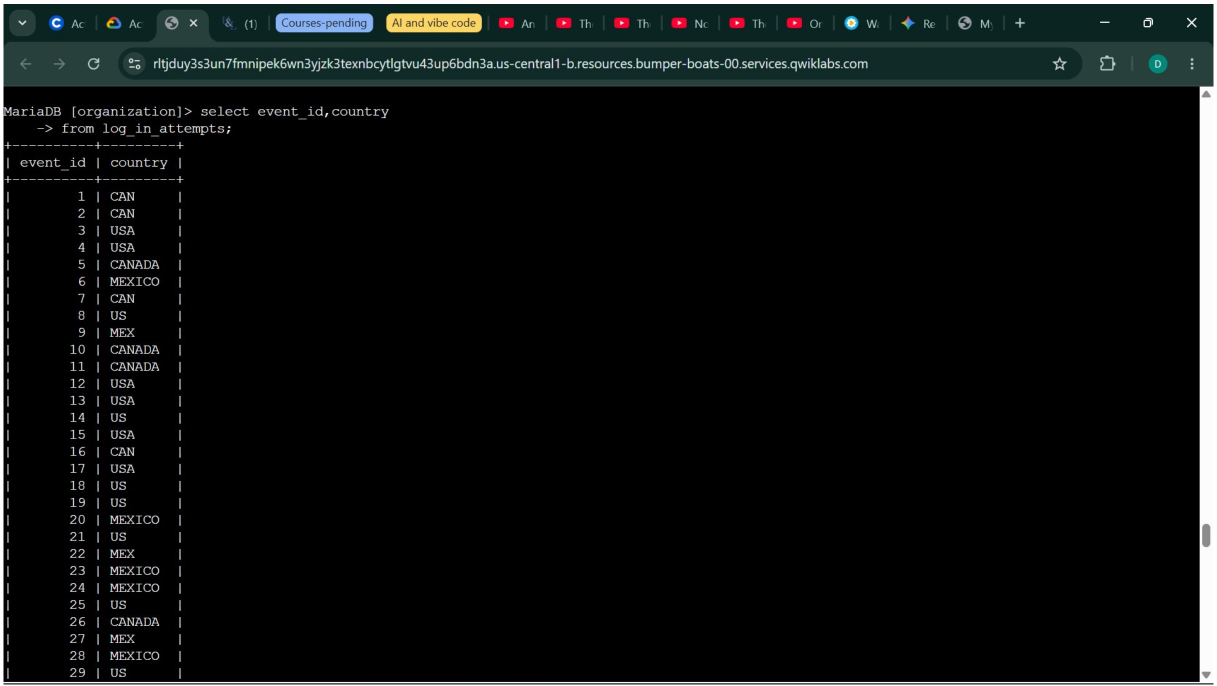
Task 2 — Investigate Login Activity

The **log_in_attempts** table was queried to look for geographically anomalous or after-hours login activity. The table contains event IDs, usernames, login timestamps, countries, IP addresses, and a success flag.

Step 1 – Check login countries

Selecting only `event_id` and `country` allows quick scanning for unexpected regions. All entries show USA/US, CAN/CANADA, MEX/MEXICO, or similar North-American variants — confirming **no logins from Australia** or other unexpected countries.

```
SELECT event_id, country
FROM log_in_attempts;
```



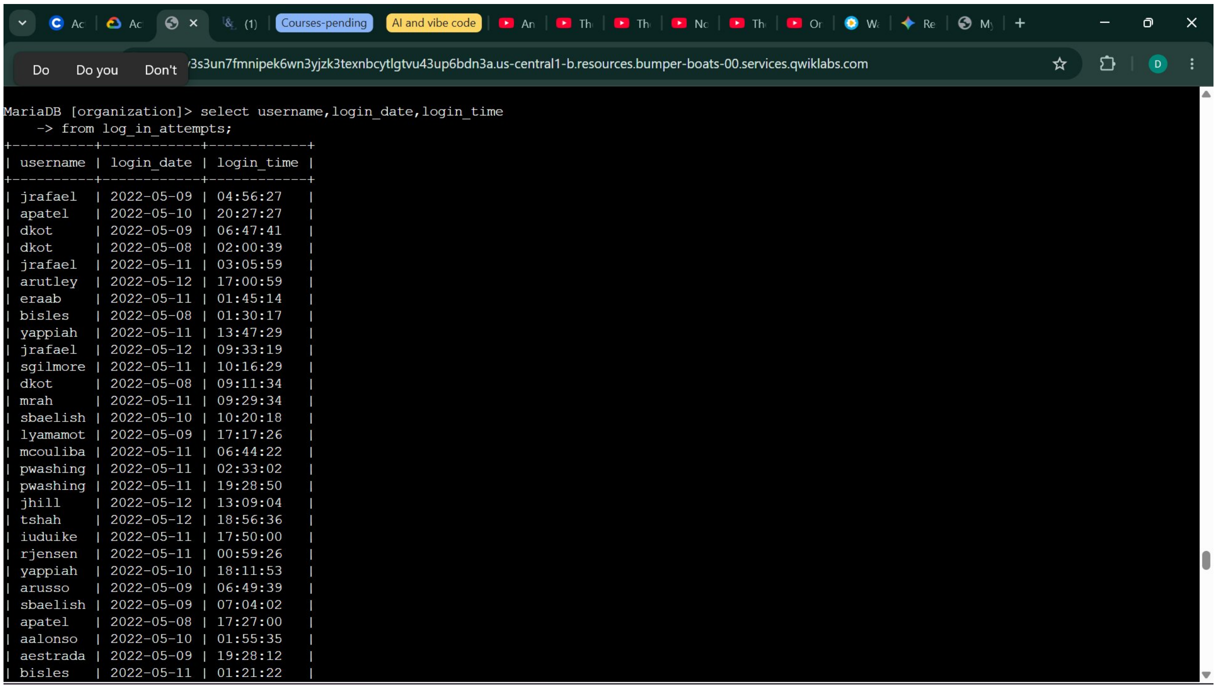
```
MariaDB [organization]> select event_id,country
    -> from log_in_attempts;
+----------+---------+
| event_id | country |
+----------+---------+
|        1 | CAN     |
|        2 | CAN     |
|        3 | USA     |
|        4 | USA     |
|        5 | CANADA  |
|        6 | MEXICO  |
|        7 | CAN     |
|        8 | US      |
|        9 | MEX     |
|       10 | CANADA  |
|       11 | CANADA  |
|       12 | USA     |
|       13 | USA     |
|       14 | US      |
|       15 | USA     |
|       16 | CAN     |
|       17 | USA     |
|       18 | US      |
|       19 | US      |
|       20 | MEXICO  |
|       21 | US      |
|       22 | MEX     |
|       23 | MEXICO  |
|       24 | MEXICO  |
|       25 | US      |
|       26 | CANADA  |
|       27 | MEX     |
|       28 | MEXICO  |
|       29 | US      |
```

Figure 4 – event_id and country columns from log_in_attempts

Step 2 – Inspect login timestamps

Selecting username, login_date, and login_time together allows identification of logins outside normal business hours. The fifth row returns **jrafael**, logged in at 03:05:59 on 2022-05-11 — a potentially suspicious early-morning access.

```
SELECT username, login_date, login_time
FROM log_in_attempts;
```



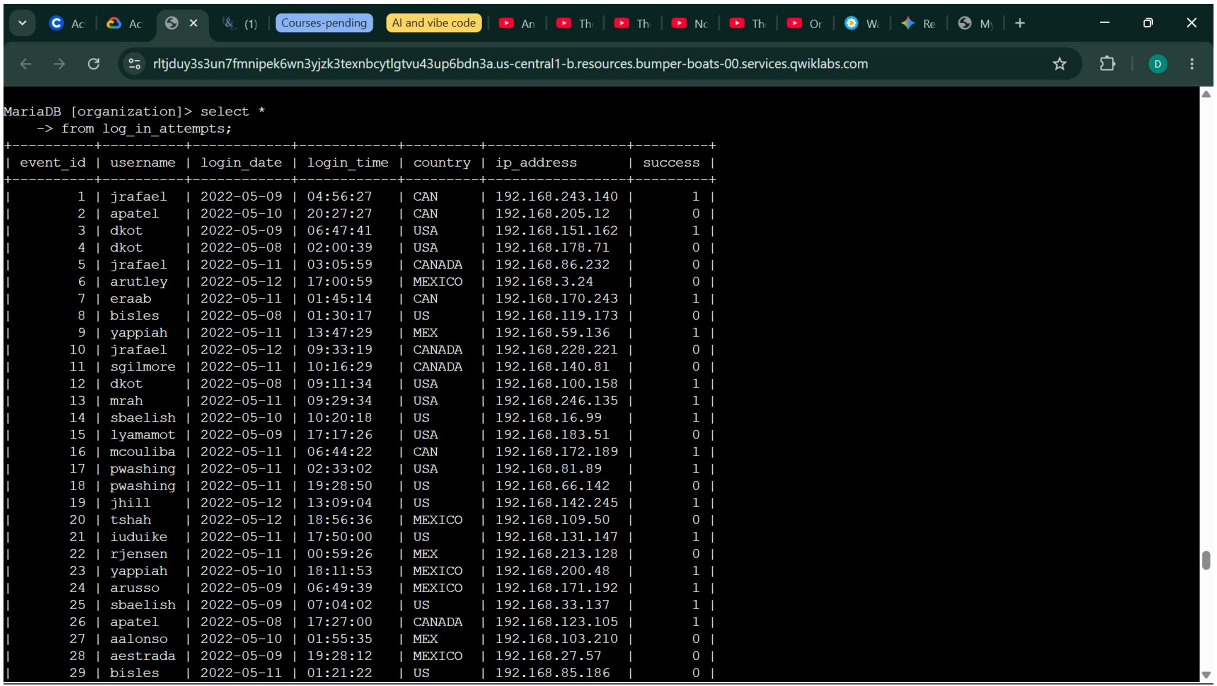
```
MariaDB [organization]> select username,login_date,login_time
    -> from log_in_attempts;
+----------+------------+------------+
| username | login_date | login_time |
+----------+------------+------------+
| jrafael  | 2022-05-09 | 04:56:27   |
| apatel   | 2022-05-10 | 20:27:27   |
| dkot     | 2022-05-09 | 06:47:41   |
| dkot     | 2022-05-08 | 02:00:39   |
| jrafael  | 2022-05-11 | 03:05:59   |
| arutley  | 2022-05-12 | 17:00:59   |
| eraab    | 2022-05-11 | 01:45:14   |
| bisles   | 2022-05-08 | 01:30:17   |
| yappiah  | 2022-05-11 | 13:47:29   |
| jrafael  | 2022-05-12 | 09:33:19   |
| sgilmore | 2022-05-11 | 10:16:29   |
| dkot     | 2022-05-08 | 09:11:34   |
| mrah     | 2022-05-11 | 09:29:34   |
| sbaelish | 2022-05-10 | 10:20:18   |
| lyamamot | 2022-05-09 | 17:17:26   |
| mcouliba | 2022-05-11 | 06:44:22   |
| pwashing | 2022-05-11 | 02:33:02   |
| pwashing | 2022-05-11 | 19:28:50   |
| jhill    | 2022-05-12 | 13:09:04   |
| tshah    | 2022-05-12 | 18:56:36   |
| iuduike  | 2022-05-11 | 17:50:00   |
| rjensen  | 2022-05-11 | 00:59:26   |
| yappiah  | 2022-05-10 | 18:11:53   |
| arusso   | 2022-05-09 | 06:49:39   |
| sbaelish | 2022-05-09 | 07:04:02   |
| apatel   | 2022-05-08 | 17:27:00   |
| aalonso  | 2022-05-10 | 01:55:35   |
| aestrada | 2022-05-09 | 19:28:12   |
| bisles   | 2022-05-11 | 01:21:22   |
```

Figure 5 – username, login_date, and login_time columns

Step 3 – Retrieve all login attempt columns

Using `SELECT *` returns the complete record for every login attempt, including the IP address and success flag — essential context for a full security review.

```
SELECT *
FROM log_in_attempts;
```



```
MariaDB [organization]> select *
    -> from log_in_attempts;
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
5	jrafael	2022-05-11	03:05:59	CANADA	192.168.86.232	0
6	arutley	2022-05-12	17:00:59	MEXICO	192.168.3.24	0
7	eraab	2022-05-11	01:45:14	CAN	192.168.170.243	1
8	bisles	2022-05-08	01:30:17	US	192.168.119.173	0
9	yappiah	2022-05-11	13:47:29	MEX	192.168.59.136	1
10	jrafael	2022-05-12	09:33:19	CANADA	192.168.228.221	0
11	sgilmore	2022-05-11	10:16:29	CANADA	192.168.140.81	0
12	dkot	2022-05-08	09:11:34	USA	192.168.100.158	1
13	mrah	2022-05-11	09:29:34	USA	192.168.246.135	1
14	sbaelish	2022-05-10	10:20:18	US	192.168.16.99	1
15	lyamamot	2022-05-09	17:17:26	USA	192.168.183.51	0
16	mcouliba	2022-05-11	06:44:22	CAN	192.168.172.189	1
17	pwashing	2022-05-11	02:33:02	USA	192.168.81.89	1
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
19	jhill	2022-05-12	13:09:04	US	192.168.142.245	1
20	tshah	2022-05-12	18:56:36	MEXICO	192.168.109.50	0
21	iuduike	2022-05-11	17:50:00	US	192.168.131.147	1
22	rjensen	2022-05-11	00:59:26	MEX	192.168.213.128	0
23	yappiah	2022-05-10	18:11:53	MEXICO	192.168.200.48	1
24	arusso	2022-05-09	06:49:39	MEXICO	192.168.171.192	1
25	sbaelish	2022-05-09	07:04:02	US	192.168.33.137	1
26	apatel	2022-05-08	17:27:00	CANADA	192.168.123.105	1
27	aalonso	2022-05-10	01:55:35	MEX	192.168.103.210	0
28	aestrada	2022-05-09	19:28:12	MEXICO	192.168.27.57	0
29	bisles	2022-05-11	01:21:22	US	192.168.85.186	0

*Figure 6 – All columns from log_in_attempts (SELECT *)*

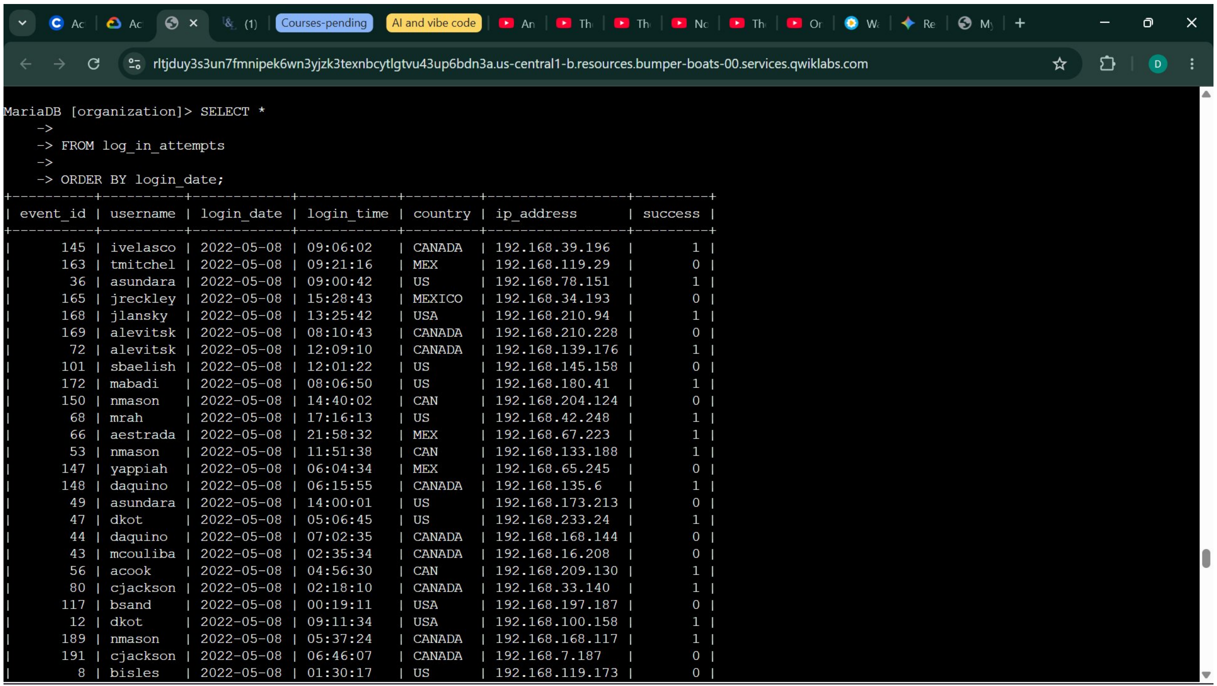
Task 3 — Order Login Attempts Data

Sorting the login data chronologically enables faster identification of the earliest activity in the dataset, which is often where initial-access attempts originate.

Step 1 – Order by login_date

Adding `ORDER BY login_date` sorts results ascending by date. The first record returned is **ivelasco on 2022-05-08**.

```
SELECT *
FROM log_in_attempts
ORDER BY login_date;
```



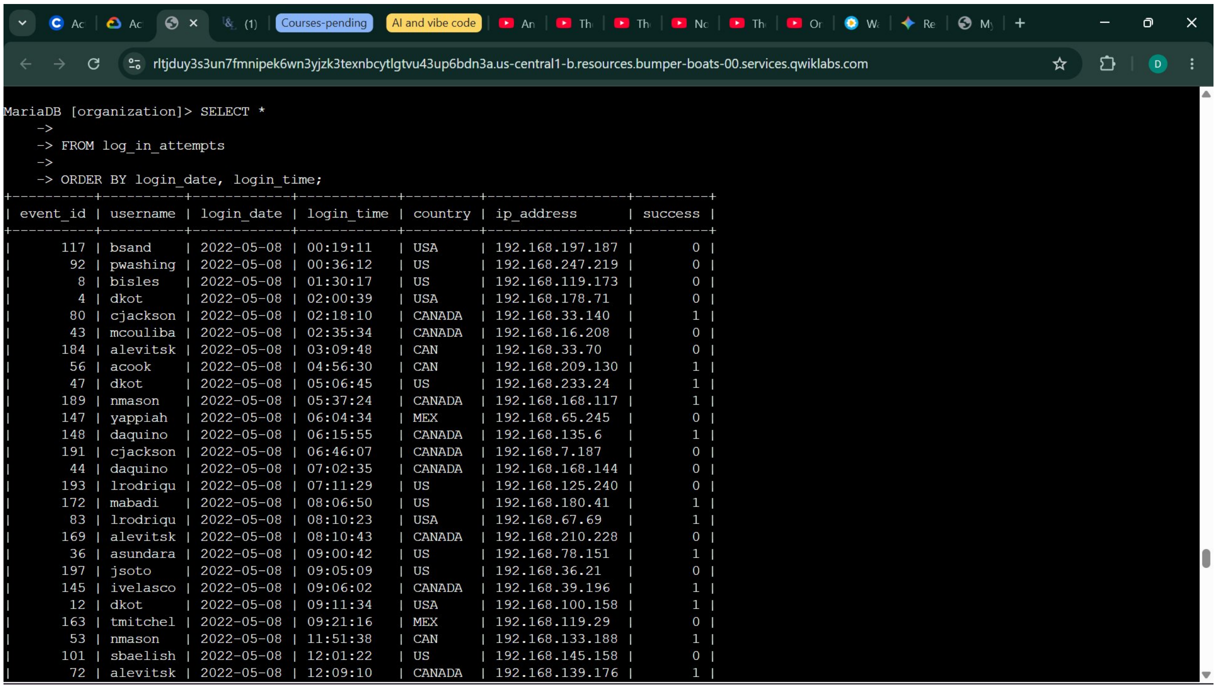

```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> ORDER BY login_date;
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|      145 | ivelasco | 2022-05-08 | 09:06:02   | CANADA  | 192.168.39.196  |       1 |
|      163 | tmitchel | 2022-05-08 | 09:21:16   | MEX     | 192.168.119.29  |       0 |
|       36 | asundara | 2022-05-08 | 09:00:42   | US      | 192.168.78.151  |       1 |
|      165 | jreckley | 2022-05-08 | 15:28:43   | MEXICO  | 192.168.34.193  |       0 |
|      168 | jlansky  | 2022-05-08 | 13:25:42   | USA     | 192.168.210.94  |       1 |
|      169 | alevitsk | 2022-05-08 | 08:10:43   | CANADA  | 192.168.210.228 |       0 |
|       72 | alevitsk | 2022-05-08 | 12:09:10   | CANADA  | 192.168.139.176 |       1 |
|      101 | sbaelish | 2022-05-08 | 12:01:22   | US      | 192.168.145.158 |       0 |
|      172 | mabadi   | 2022-05-08 | 08:06:50   | US      | 192.168.180.41  |       1 |
|      150 | nmason   | 2022-05-08 | 14:40:02   | CAN     | 192.168.204.124 |       0 |
|       68 | mrah     | 2022-05-08 | 17:16:13   | US      | 192.168.42.248  |       1 |
|       66 | aestrada | 2022-05-08 | 21:58:32   | MEX     | 192.168.67.223  |       1 |
|       53 | nmason   | 2022-05-08 | 11:51:38   | CAN     | 192.168.133.188 |       1 |
|      147 | yappiah  | 2022-05-08 | 06:04:34   | MEX     | 192.168.65.245  |       0 |
|      148 | daquino  | 2022-05-08 | 06:15:55   | CANADA  | 192.168.135.6   |       1 |
|       49 | asundara | 2022-05-08 | 14:00:01   | US      | 192.168.173.213 |       0 |
|       47 | dkot     | 2022-05-08 | 05:06:45   | US      | 192.168.233.24  |       1 |
|       44 | daquino  | 2022-05-08 | 07:02:35   | CANADA  | 192.168.168.144 |       0 |
|       43 | mcouliba | 2022-05-08 | 02:35:34   | CANADA  | 192.168.16.208  |       0 |
|       56 | acook    | 2022-05-08 | 04:56:30   | CAN     | 192.168.209.130 |       1 |
|       80 | cjackson | 2022-05-08 | 02:18:10   | CANADA  | 192.168.33.140  |       1 |
|      117 | bsand    | 2022-05-08 | 00:19:11   | USA     | 192.168.197.187 |       0 |
|       12 | dkot     | 2022-05-08 | 09:11:34   | USA     | 192.168.100.158 |       1 |
|      189 | nmason   | 2022-05-08 | 05:37:24   | CANADA  | 192.168.168.117 |       1 |
|      191 | cjackson | 2022-05-08 | 06:46:07   | CANADA  | 192.168.7.187   |       0 |
|        8 | bisles   | 2022-05-08 | 01:30:17   | US      | 192.168.119.173 |       0 |
```

Figure 7 – log_in_attempts ordered by login_date

Step 2 – Order by login_date, then login_time

Adding a secondary sort key (`login_time`) breaks ties within each date. The earliest event in the entire dataset becomes **bsand at 00:19:11 on 2022-05-08** — the first chronological login attempt recorded.

```
SELECT *
FROM log_in_attempts
ORDER BY login_date, login_time;
```



```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> ORDER BY login_date, login_time;
```

event_id	username	login_date	login_time	country	ip_address	success
117	bsand	2022-05-08	00:19:11	USA	192.168.197.187	0
92	pwashing	2022-05-08	00:36:12	US	192.168.247.219	0
8	bisles	2022-05-08	01:30:17	US	192.168.119.173	0
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
80	cjackson	2022-05-08	02:18:10	CANADA	192.168.33.140	1
43	mcouliba	2022-05-08	02:35:34	CANADA	192.168.16.208	0
184	alevitsk	2022-05-08	03:09:48	CAN	192.168.33.70	0
56	acook	2022-05-08	04:56:30	CAN	192.168.209.130	1
47	dkot	2022-05-08	05:06:45	US	192.168.233.24	1
189	nmason	2022-05-08	05:37:24	CANADA	192.168.168.117	1
147	yappiah	2022-05-08	06:04:34	MEX	192.168.65.245	0
148	daquino	2022-05-08	06:15:55	CANADA	192.168.135.6	1
191	cjackson	2022-05-08	06:46:07	CANADA	192.168.7.187	0
44	daquino	2022-05-08	07:02:35	CANADA	192.168.168.144	0
193	lrodriqu	2022-05-08	07:11:29	US	192.168.125.240	0
172	mabadi	2022-05-08	08:06:50	US	192.168.180.41	1
83	lrodriqu	2022-05-08	08:10:23	USA	192.168.67.69	1
169	alevitsk	2022-05-08	08:10:43	CANADA	192.168.210.228	0
36	asundara	2022-05-08	09:00:42	US	192.168.78.151	1
197	jsoto	2022-05-08	09:05:09	US	192.168.36.21	0
145	ivelasco	2022-05-08	09:06:02	CANADA	192.168.39.196	1
12	dkot	2022-05-08	09:11:34	USA	192.168.100.158	1
163	tmitchel	2022-05-08	09:21:16	MEX	192.168.119.29	0
53	nmason	2022-05-08	11:51:38	CAN	192.168.133.188	1
101	sbaelish	2022-05-08	12:01:22	US	192.168.145.158	0
72	alevitsk	2022-05-08	12:09:10	CANADA	192.168.139.176	1

Figure 8 – log_in_attempts ordered by login_date and login_time

Summary

This lab reinforced three foundational SQL skills directly applicable to security analysis work:

- **Column selection** — Using named columns in `SELECT` reduces noise and surfaces only the fields needed for a specific investigation.
- **Wildcard retrieval** — `SELECT *` provides a complete record snapshot when all fields are relevant, such as during a full audit.
- **Chronological ordering** — `ORDER BY` with one or more date/time columns is critical for reconstructing timelines during incident response, making it straightforward to find the earliest or latest events in a log.

No anomalous geographic logins (e.g. from Australia) were found. However, several after-hours login attempts were noted (e.g. jrafael at 03:05 and bisles at 01:30), warranting further investigation with SQL filter conditions in subsequent labs.